1.- B. Aunque funciona con las colecciones estándar que implementan Iterable (como List o Set) y con matrices, no se puede usar con cualquier estructura de datos del framework de colecciones, siendo el ejemplo clásico el Map, el cual no implementa Iterable de forma directa y requiere invocar primero a .entrySet(), .keySet() o .values().
2.- Se evalúa la primera condición del bloque if: ¿a == b? (10 == 37). Esto es falso, por lo que no entra a ese bloque y z sigue valiendo 0.
Se evalúa la siguiente condición del bloque else if: ¿a > b? (10 > 37). Esto también es falso, por lo que tampoco entra y z se mantiene en 0.
Finalmente, se calcula w = 10 * z, lo que resulta en 10 * 0 = 0.
3.- D. i = 0: Par (0 % 2 == 0) → count aumenta a 1
i = 1: Impar
i = 2: Par → count aumenta a 2
i = 3: Impar
i = 4: Par → count aumenta a 3
i = 5: Impar
i = 6: Par → count aumenta a 4
i = 7: Impar
i = 8: Par → count aumenta a 5
i = 9: Impar
4.- A. El ciclo externo (i) controla las filas y se repetirá 3 veces (i toma los valores 1, 2 y 3). Para cada valor individual de i, el ciclo interno (j) se ejecuta por completo desde el principio hasta el final (j toma los valores 1 y 2).

5.- C. **Ciclo externo (i)**: Se ejecuta **4 veces** (cuando i vale 0, 1, 2 y 3).
1. **Ciclo intermedio (j)**: Se ejecuta **3 veces** por cada vuelta del ciclo externo (cuando j vale 0, 1 y 2).
2. **Ciclo interno (k)**: Se ejecuta **2 veces** por cada vuelta del ciclo intermedio (cuando k vale 0 y 1).

Multiplicando todas las iteraciones:

$$4 \times 3 \times 2 = 24$$

6.- D. en cada iteración `ii` seguirá valiendo 2, la condición `ii > 0` siempre será verdadera, y el programa imprimirá el número 2 de forma **infinita** hasta que detengas el proceso o se quede sin memoria.
7.- B. La expresión --ii decrementa la variable y devuelve un valor de tipo int (1, 0, -1, etc.). Como un int no se puede convertir automáticamente a boolean en Java, el compilador arroja un error de tipo (type mismatch).
8.- E. Buscamos que el ciclo se ejecute exactamente 5 veces para que la variable jj (que empieza en 0 y aumenta de 1 en 1 con j++) termine valiendo 5.
El ciclo se mantiene ejecutándose mientras ii > 6 y en cada vuelta se decrementa en 1 (ii -= 1).
9.- El bucle `for` recorre el arreglo `arr = {1, 2, 3, 4, 5}` desde el índice `i = 0` hasta `4`:

- **`i = 0`** (índice par): `0 % 2 == 0` se cumple, por lo que se suma `arr[0]` (1).
  - `num = 0 + 1 = 1`

- **`i = 1`** (índice impar): `1 % 2 == 0` no se cumple, por lo que se resta `arr[1]` (2).
  - `num = 1 - 2 = -1`
- **`i = 2`** (índice par): `2 % 2 == 0` se cumple, se suma `arr[2]` (3).
  - `num = -1 + 3 = 2`
- **`i = 3`** (índice impar): `3 % 2 == 0` no se cumple, se resta `arr[3]` (4).
  - `num = 2 - 4 = -2`
- **`i = 4`** (índice par): `4 % 2 == 0` se cumple, se suma `arr[4]` (5).
  - `num = -2 + 5 = 3`

Al finalizar el bucle, `System.out.println(num);` imprime **`3`**.

10.- A. Prueba de escritorio

11.- En Java, cuando una sentencia switch coincide con un case y no tiene una instrucción break, la ejecución continúa de forma secuencial evaluando e imprimiendo los siguientes casos (fall-through behavior), sin importar si cumplen la condición o no, hasta encontrar un break o salir del switch.

12.- E. Pasa lo mismo de la pregunta 11.

13.- F. **Comportamiento de *Fall-through***: Al igual que en los ejemplos anteriores, **no hay ningún `break`** en todo el bloque del `switch`. Esto significa que una vez que entra en el caso coincidente, la ejecución fluye de manera continua hacia abajo por todos los casos restantes:

- **`case 'B':`** Imprime **`B`** y continúa al siguiente sin detenerse.
- **`case 'C':`** Imprime **`C`** y continúa.
- **`case d:`** **(D) y `case 'F':`** Llega hasta la impresión de `F`, imprimiendo **`F`**.

14.- F. **Comportamiento de *Fall-through***: Como **no hay un `break`** inmediatamente después de imprimir `"B"`, el flujo continúa hacia abajo de forma secuencial al siguiente caso, que es **`case 'C':`**.

15.-B. **El bucle `while (x++ < 5)`**:

- El operador de post-incremento (`x++`) evalúa la condición usando el valor **actual** de `x` y luego incrementa `x` en 1.
- **Iteración 1**: Evalúa `0 < 5` (verdadero) → `x` se incrementa a `1`.
- **Iteración 2**: Evalúa `1 < 5` (verdadero) → `x` se incrementa a `2`.
- **Iteración 3**: Evalúa `2 < 5` (verdadero) → `x` se incrementa a `3`.
- **Iteración 4**: Evalúa `3 < 5` (verdadero) → `x` se incrementa a `4`.
- **Iteración 5**: Evalúa `4 < 5` (verdadero) → `x` se incrementa a `5`.
- **Iteración 6**: Evalúa `5 < 5` (**falso**) → El bucle se detiene, pero `x` de todas formas se incrementa a **`6`** durante la evaluación de la condición.

16.- **Variables iniciales**:

- table tiene 3 elementos (length = 3).
- ii = 0 (declarada **fuera** de ambos bucles, por lo que su valor se conserva entre iteraciones).

Primera iteración del bucle for (ss = "aa"):

- El bucle while evalúa ii < 3 ($0 < 3$, verdadero).
- Imprime ii (**0**).
- Incrementa ii++ → ii pasa a ser **1**.
- El break sale del bucle while.

Segunda iteración del bucle for (ss = "bb"):

- El bucle while evalúa ii < 3 ($1 < 3$, verdadero).
- Imprime ii (**1**).
- Incrementa ii++ → ii pasa a ser **2**.
- El break sale del bucle while.

Tercera iteración del bucle for (ss = "cc"):

- El bucle while evalúa ii < 3 ($2 < 3$, verdadero).
- Imprime ii (**2**).
- Incrementa ii++ → ii pasa a ser **3**.
- El break sale del bucle while.

17.- Valor inicial de los campos estáticos: En Java, una variable booleana declarada a nivel de clase (static boolean b;) se inicializa por defecto con el valor false.

18.- D. En Java, la condición de un bucle while (...) debe evaluar estrictamente a un tipo booleano (boolean o Boolean). Si se coloca cualquier otro tipo de dato (como un entero), el compilador arrojará un error de tipo (incompatible types).

19.- A. En Java, el bucle for tiene la estructura: for (inicialización; condición; actualización) { cuerpo }. Es fundamental recordar el orden exacto en el que se ejecutan sus partes:

20.- La variable j se declara e inicializa en 0 fuera del bucle for principal, lo que significa que su valor se acumula y no se reinicia en cada iteración de i. Las opciones de inicialización de x, y y z permiten que los bucles anidados (for, do-while y while) se ejecuten de manera controlada sin generar bucles infinitos o fallas de lógica en la evaluación de las condiciones de salida.